

Hackathon Problem Statements

→ The Dropout Signal

Problem context

Universities lose students to dropout every year — but the warning signs appear months earlier. Declining grades, missed assignments, and financial stress all precede the decision to leave. The challenge is not just prediction: it is fair prediction.

A model that disproportionately flags students from lower-income backgrounds or specific demographics is a liability, not a tool. Your pipeline must predict risk early, explain why, and prove it is not biased against any group.

Dataset — pre-loaded in your cluster

UCI Dropout Dataset [start here]

- **Path:** /Volumes/hackbricks/raw/uci_dropout.csv
- **Size:** 4,400 records, 37 features — trains in seconds
- **Target:** Use binary: Dropout = 1, Enrolled/Graduate = 0
- **Contains:** Demographics, semester grades, financial indicators, macroeconomic context
- **Auth:** None — direct access, no login required

OULAD [only if you have spare time after hour 14]

- **Path:** /Volumes/hackbricks/raw/oulad/
- **Size:** ~32,000 students, 7 interlinked tables
- **Warning:** Multi-table join takes 3–4 hours alone. Skip unless your core pipeline is done and clock says < hour 14.

Enrichment sources — optional, after hour 14 only

- AISHE India Education Data: <https://aishe.gov.in> — district-level dropout rates, India context
- World Bank Education Stats: <https://datatopics.worldbank.org/education/> — macroeconomic context
- Kaggle education datasets: <https://www.kaggle.com/datasets?search=student+dropout>

What you must build — in priority order

Build in this order. Stop when time runs out. A clean 4-step pipeline with a documented fairness audit beats a 7-step pipeline that never reaches fairness.

1. **Bronze layer:** Ingest `uci_dropout.csv` into a Bronze Delta table. Document schema.
2. **Silver layer:** Clean nulls, engineer features: semester-on-semester grade delta, absenteeism trend, financial stress index.
3. **Two models:** Logistic Regression baseline + one tree model (Random Forest or XGBoost). Log both runs in MLflow. Register the better one in Model Registry.
4. **Fairness audit:** Compute demographic parity and equal opportunity across gender and socioeconomic group. Log metrics as a Delta table. Document any disparity found — do not hide it.
5. **SHAP explainability:** Run SHAP on your best model. Surface top 3 risk factors per flagged student.
6. **Gold output table:** At-risk students: student ID, risk score, top 3 factors, recommended intervention tier (low / medium / high).
7. **[Bonus] OULAD enrichment:** If time allows after hour 14: join VLE engagement logs to add clickstream-based engagement features.

Databricks stack

- Delta Lake — Bronze / Silver / Gold medallion architecture
- MLflow — experiment tracking, model registry

- Unity Catalog — Gold table registration
- SHAP — pip install shap (explainability)
- Databricks AutoML — optional bonus benchmark

Evaluation criteria

Criteria	Weight
Pipeline architecture — Bronze / Silver / Gold discipline	20%
Feature engineering depth & creativity	20%
Model performance & MLflow experiment hygiene	20%
Fairness audit — metrics, documentation, honest findings	25%
Explainability & Gold table output quality	15%

→ The Drifting Oracle

Problem Context

A credit scoring model trained on historical data is now scoring applicants in a post-inflation economy — feature distributions have shifted and nobody noticed. Simultaneously, an LLM explaining loan rejections in plain English has started generating plausible-sounding but legally non-existent justifications. Both problems need monitoring systems, not patches.

Datasets

Name	Link	Details
Home Credit Default Risk	Kaggle	350K loan applications, rich feature set — use as training distribution
German Credit Dataset	UCI ML Repository	1K records, different feature distribution — use as simulated drift batch
RBI / SEBI Guidelines Reference	RBI Notifications	Provide teams a curated list of real regulations as ground truth for LLM eval

🎯 Objectives

1. **Baseline Model** — Train a credit risk classifier on Home Credit data; log all params, metrics, and artifacts with MLflow; register as `Champion` in Model Registry
2. **Drift Detection** — Score the German Credit batch; compute PSI (Population Stability Index) on at least 5 key features; write drift metrics to a Delta table
3. **Retraining Trigger** — When PSI exceeds threshold on any feature, trigger automated retraining; register new model as `Challenger`; compare Champion vs Challenger using `mlflow.evaluate()`
4. **LLM Evaluation Pipeline** — Build a pipeline using `mlflow.evaluate()` with custom metrics that score each loan rejection explanation for:
 - Factual grounding (does it reference actual data?)
 - Hallucination (does it cite a regulation not in the reference list?)
5. **Governance** — All monitoring outputs (drift metrics, eval scores, retraining logs) must land in governed Unity Catalog Delta tables

🔧 Databricks Stack

- MLflow (experiment tracking, model registry, `mlflow.evaluate()`)
- Delta Lake (feature store, monitoring tables)
- Databricks Model Serving (LLM endpoint)
- Unity Catalog (governance of monitoring outputs)

🏆 Evaluation Criteria

Criteria	Weight
Drift detection methodology & PSI implementation	25%
Retraining trigger logic & Champion/Challenger comparison	25%
LLM eval pipeline with custom hallucination metrics	25%
MLflow experiment hygiene & registry discipline	15%
Unity Catalog governance of all outputs	10%

💬 **Evaluator Note:** Push teams on their PSI threshold choice — why that number? Also ask how their LLM eval would scale to 10,000 explanations per day.

→ Fire Risk Prevention & Response Optimization Engine

Complexity Level: Advanced

Real-World Application: City of San Francisco Fire Department Operations

Project Objective

Build an integrated data platform that synthesizes 14 days of analysis into a production-ready system supporting fire department strategic planning, proactive compliance, and resource optimization.

Capstone Deliverables

1. **Risk Prediction Layer** - Predictive models identifying high-risk properties and neighborhoods
2. **Response Optimization Layer** - Data-driven resource allocation recommendations
3. **Compliance Tracking System** - Violation follow-up effectiveness measurement
4. **Operational Dashboard Dataset** - Aggregated metrics for decision-makers

Expected Outcomes

- 5+ optimized Delta tables in Gold layer
- Comprehensive technical documentation
- Executive summary with actionable recommendations
- Code repository and reproducible pipeline

BLACKBRICKS

Eval Round 1

Team Name:

Key Criteria	Description	Marks (1–10)
Problem Understanding	Clarity in understanding the assigned domain/problem, background research, and relevance of the problem they are solving.	
Innovation & Creativity	How original is the solution idea? Does it show out-of-the-box thinking or a fresh approach to an existing challenge?	
Feasibility & Planning	How practical is the proposed approach? Is there a clear plan or roadmap that shows realistic implementation steps?	
Team Coordination	Evidence of good collaboration: clear role distribution, communication, and planning among team members.	

Comments:

Eval Round 2

Team Name:

Criteria	Description	Marks (1–10)
Technical Execution	Skill and sophistication shown in building the solution (tech stack, efficiency, clean code, appropriate tools).	
Progress & Completeness	How much of the solution has been implemented? Are the key features functional or demonstrated?	
Problem Alignment	Does the solution still solve the problem effectively? Are changes or pivots well-justified?	
Functionality & Usability	How well does the product work for end users? Is the interface clear and the experience usable?	

Comments:

EVAL ROUND 3

Team Name :

Criteria	Description	Marks (1-10)
Project Completion & Polish	Is the final solution complete, stable, and refined? Are all core components working smoothly?	
Impact & Value	What is the real-world impact potential? Could this be scaled or deployed to benefit a specific domain or audience?	
Presentation & Communication	Was the idea clearly explained? Did the team effectively communicate their journey, learnings, and solution in the final demo?	
Team Growth & Resilience	How did the team handle feedback, setbacks, or technical hurdles throughout the hackathon?	

Comments: